

Introduction to DL & GenAI Project

Author: Hariom Patel

Roll number: 24f1000881

1. Overview

This project tackles the **Smart MCQ Solver Challenge** on Kaggle, where each question has a text **prompt** and five options (A–E), and the goal is to predict the **top three most likely answers in ranked order**. The evaluation metric is **Mean Average Precision at 3 (MAP@3)**, which rewards placing the correct option earlier in the prediction list.

The work explores a progression from **classical NLP** to **deep learning**:

- Model 0: TF-IDF + Logistic Regression (baseline).
- Model 1: BiGRU + Attention (custom RNN from scratch).
- Model 2: ELECTRA-base Multiple-Choice Transformer (pretrained).
- Model 3: TF-IDF + PyTorch MLP (deep neural network over sparse features).

All experiments use PyTorch, Hugging Face Transformers, scikit-learn, and Weights & Biases (W&B) for tracking metrics and saving models.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge provides a dataset of multiple-choice questions, each with a textual prompt and five answer options (A–E). The task is to predict **three labels in ranked order** for each question; the evaluation metric is **Mean Average Precision at 3 (MAP@3)**.

2.2 Objectives

- Build efficient MCQ ranking models that can achieve high MAP@3 on validation and Kaggle.
- Experiment with different architectures: a classical baseline, a custom deep model, and a pretrained transformer.
- Integrate Weights & Biases for experiment tracking and use PyTorch / Hugging Face Transformers effectively.
- Analyse the trade-off between performance, complexity, and training time for each approach

3. Dataset and Preprocessing

3.1 Data Description

The competition provides three CSV files:

- train.csv: 2000 questions, columns id, prompt, A, B, C, D, E, answer.
- test.csv: 500 questions, columns id, prompt, A, B, C, D, E.
- sample_submission.csv: ID, Prediction with three labels.

Column checks and `isnull().sum()` show **no missing values** in train or test, so no imputation was required.

3.2 Quick EDA

Prompt statistics from the notebook:

- Average question length $\approx$ **18 words**, $\approx$ **118 characters**.
- Shortest question: **3** words; longest: **51** words.
- Average unique words per question $\approx$ **16**.

Label distribution for answer (A–E) is reasonably balanced; the coefficient of variation of counts is small, so there is no serious class imbalance.

3.3 Long-Format Data for Ranking

To model ranking properly, a **long-format dataframe** is constructed:

- Five rows per question, one for each option.
- Columns:
 - id – question ID
 - prompt – question text
 - option_label – A–E
 - option_text – option content
 - combined_text – "Question: <prompt> [SEP] Option: <option_text>"
 - target – 1 if this option is correct, 0 otherwise

The training long-format has **7030 rows** and is used by TF-IDF, BiGRU, and MLP models.

3.4 Train–Validation Split

Unique question IDs are split into training and validation sets:

- Train: **1406** questions
- Validation: **352** questions

Long-format datasets are built separately for each split, so all models share the same validation set.

4. Features and Tokenization

4.1 TF-IDF Features

For classical models and the MLP:

- `TfidfVectorizer(max_features=30000, ngram_range=(1,2), stop_words='english')`.
- Applied to `combined_text` in the long-format dataset.
- Output:
 - Sparse TF-IDF matrices for Logistic Regression.
 - Dense tensors (via a sparse-to-dense helper) for the PyTorch MLP.

4.2 Scratch Tokenizer for BiGRU

For the BiGRU model:

- Tokenization: split `combined_text` on whitespace.
- Vocabulary: tokens with frequency ≥ 2 , plus `PAD = 0` and `UNK = 1`.
- Encoding:
 - Map tokens to indices.
 - Truncate or pad sequences to **length 128**.

4.3 ELECTRA Tokenizer for Multiple Choice

For the ELECTRA transformer:

- Tokenizer: `AutoTokenizer.from_pretrained("google/electra-base-discriminator")`.
 - For each question and option:
 - Sentence 1: Question: `<prompt>` the most appropriate answer.
 - Sentence 2: Candidate Answer: `<option_text>`.
 - Five (question, option) pairs per question.
 - `max_length = 320`, with truncation and padding; attention masks (and token type IDs when available)
-

5. Models and Training

5.1 Model 0 – TF-IDF + Logistic Regression

Idea: Simple, strong baseline using linear classification on TF-IDF features.

- Input: TF-IDF vectors of `combined_text`.

- Classifier: LogisticRegression(max_iter=MAXITER) predicting binary target.
- Ranking: For each question, compute probability for each option, sort options, return top-3 labels.

Training:

- Fit TF-IDF on training long-format data; transform train and validation.
- Train Logistic Regression on train TF-IDF features.
- Evaluate with:
 - Accuracy, macro F1, log loss via computebinarymetrics.
 - MAP@3 via maketop3predictions and mapk.

Validation (best): loss ≈ 0.34 , accuracy ≈ 0.84 , macro F1 ≈ 0.61 , MAP@3 ≈ 0.97 .

5.2 Model 1 – BiGRU + Attention (Scratch)

Idea: Custom RNN that learns from scratch to score each option using attention over tokens.

Architecture:

- Embedding: nn.Embedding(vocab_size, 128, padding_idx=0).
- BiGRU: nn.GRU(128, 128, batch_first=True, bidirectional=True) $\rightarrow$ 256-dim hidden states.
- Attention:
 - Linear layer $\rightarrow$ attention scores per time step.
 - Masked softmax $\rightarrow$ attention weights.
 - Weighted sum $\rightarrow$ context vector.
- Classifier:
 - Linear(256, 64) + BatchNorm + ReLU + Dropout(0.3).
 - Linear(64, 1) logit; sigmoid at evaluation for probability.

Training:

- Dataset: ScratchDataset with input_ids (maxlen 128) and target.
- Loss: BCEWithLogitsLoss.
- Optimizer: Adam(lr=2e-3).
- Gradient clipping: clipgradnorm(parameters, 1.0).
- Epochs: **5**, batch size: **64**.

- Logging: W&B run BiGRU-Attention-Scratch.

Validation (best epoch): Loss ≈ 0.056 , Accuracy ≈ 0.97 , Macro F1 ≈ 0.93 , MAP@3 ≈ 0.99

5.3 Model 2 – ELECTRA-Base Multiple-Choice Transformer

Idea: Use a powerful pretrained transformer designed for multiple-choice tasks.

Architecture:

- Model: AutoModelForMultipleChoice with google/electra-base-discriminator.
- Input: ElectraMCQDataset builds 5 (question, option) pairs per question.
- ELECTRA encoder processes all sequences, multiple-choice head outputs 5 logits.
- Softmax gives probabilities over labels A–E.

Training:

- Dataset: train and validation ElectraMCQDataset.
- Loss: CrossEntropyLoss(weight=class_weights) with label-based class weights.
- Optimizer: AdamW(lr=2e-5, weight_decay=0.01).
- Scheduler: get_linear_schedule_with_warmup with $\sim 10\%$ warmup steps.
- Gradient clipping: clipgradnorm(parameters, 1.0).
- Epochs: 6 (from history table), batch sizes: TRAINBATCHSIZE, VALIDBATCHSIZE.
- Logging: W&B run model2pretrainedelectrabase.

Validation (best row):

Loss ≈ 0.117 , Accuracy ≈ 0.994 , Macro F1 ≈ 0.994 , MAP@3 ≈ 0.997

5.4 Model 3 – TF-IDF + PyTorch MLP

Idea: Deep MLP on TF-IDF vectors, combining classical features with deep learning flexibility.

Architecture:

- Input: dense TF-IDF tensor (30,000 features).
- Network (CustomMLPClassifier):
 - Linear(30000 $\rightarrow$ 128) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3).
 - Linear(128 $\rightarrow$ 64) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3).

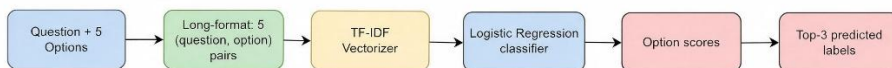
- Linear($64 \rightarrow 1$) logit; sigmoid at evaluation.

Training:

- Dataset: `TFIDFDataset(Xtraintensor, ytrain)` and `TFIDFDataset(Xvaltensor, yval)`.
- Loss: `BCEWithLogitsLoss`.
- Optimizer: `AdamW(lr=2e-3, weight_decay=0.01)`.
- Epochs: **6**, batch size: **64**.
- Logging: W&B run `model3tfidfmlpdl`.

Validation (best): Loss $\approx$ **0.058**, Accuracy $\approx$ **0.974**, Macro F1 $\approx$ **0.959**, MAP@3 $\approx$ **0.973**

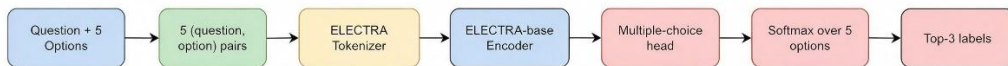
TF-IDF + Logistic Regression



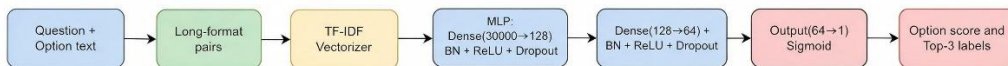
BiGRU + Attention



ELECTRA-base MCQ



TF-IDF + PyTorch MLP



6. Performance & Comparative Analysis

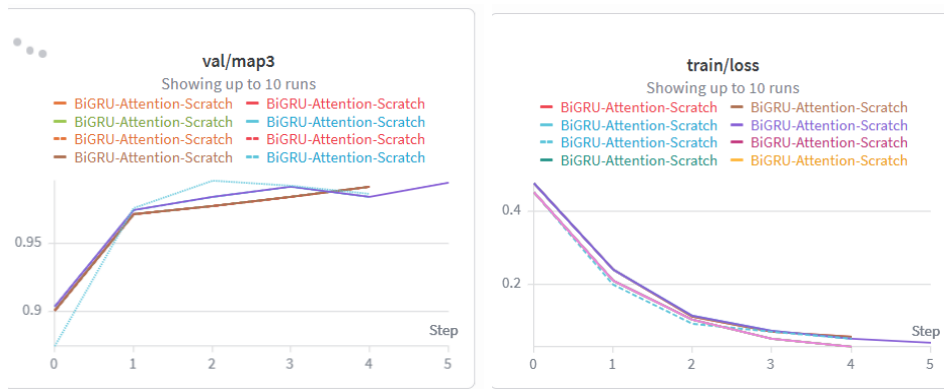
6.1 Evaluation Metrics

All models are evaluated using:

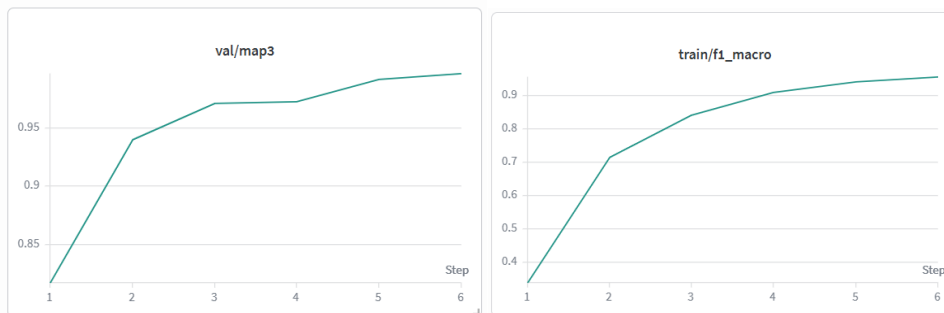
- MAP@3 (Mean Average Precision at 3) – primary Kaggle metric, computed with `apk` and `mapk` functions on top-3 ranked predictions.
- Accuracy – fraction of correctly classified options at the option level.
- Macro F1-score – average F1 across both classes (correct vs incorrect option), useful when positive labels are rare.
- Log loss – measures calibration of predicted probabilities.

6.2 Training and Validation Performance (W&B)

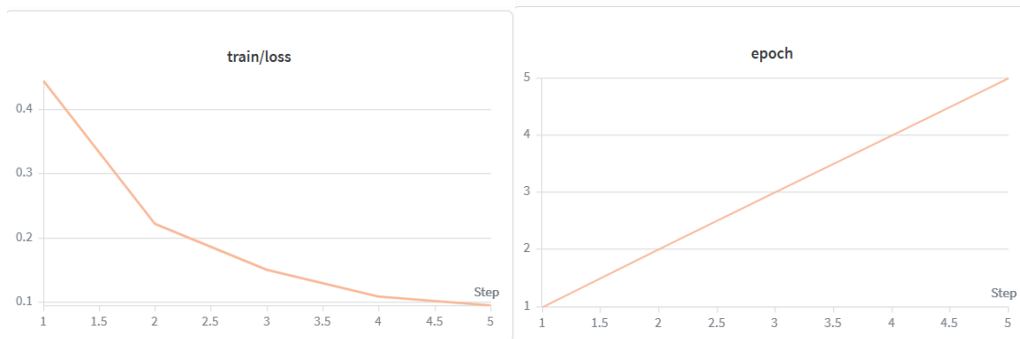
Model 1: BiGRU Attention Scratch



Model 2: ELECTRA-Base Multiple-Choice Transformer



Model 3: TF- IDF + Pytorch MLP



6.3 Comparative Metrics Table

The table below summarizes the best validation performance of each model:

Model	Features	Acc (Val)	Macro F1 (Val)	MAP@3 (Val)
Model 0 – TF-IDF + Logistic Regression	TF-IDF	0.84	0.61	0.97
Model 1 – BiGRU + Attention	Scratch tokens	0.97	0.993	0.99
Model 2 – ELECTRA MCQ	ELECTRA tokenizer	0.994	0.994	0.997

Model 3 – TF-IDF + MLP	TF-IDF	0.974	0.959	0.973
------------------------	--------	-------	-------	-------

6.4 Kaggle Performance

The final model submitted to Kaggle produced the following results:

- Kaggle Public Leaderboard MAP@3: 0.75
- Kaggle Rank: 288 (among all participants who submitted).

7. Conclusion and Future Work

Key takeaways:

- Framing MCQ solving as **option ranking with MAP@3** works well.
- **TF-IDF baselines** remain competitive and are easy to implement.
- A custom **BiGRU + attention** model can approach transformer performance with fewer parameters.
- **ELECTRA-base** provides excellent results when integrated with a proper multiple-choice head and tokenization.
- Using **W&B** makes experiment management and comparison much easier.

Future ideas:

- Try other transformers (RoBERTa, DeBERTa, T5) for multiple-choice.
- Combine models (e.g., average scores from ELECTRA and BiGRU) as an ensemble.
- Add stronger regularization or data augmentation.

8. References

1. Smart MCQ Solver Challenge – Kaggle competition page and data description.
2. Google ELECTRA-base discriminator – original paper and Hugging Face model card.
3. PyTorch documentation – GRU, Embedding, Dropout, BatchNorm, optimizers, schedulers, loss functions.
4. Hugging Face Transformers – multiple-choice models and tokenizers.
5. Scikit-learn – TfidfVectorizer, LogisticRegression, evaluation metrics.
6. Weights & Biases – experiment tracking and visualization tools.